

<주제>

토양, 기후, 작물 관련 피쳐로 관개 필요도를 예측하기

<파이프라인>

1. 전처리

- Train, test 데이터를 로드하고, train에서의 타겟이 결측치인 행을 dropna으로 제거함. 제출용 id는 따로 보관함.

2. 피쳐 엔지니어링

- Train과 test에 동일하게 적용되도록 함수화함. 4개의 파생변수를 생성함.
- Total_water_input(강수량 및 이전 관개량 합산)
- Evaporate_stress(증발 스트레스를 나타내는 상호작용 피쳐)
- Relative_crop_moisture(현재 토양 수분을 작물 종류별 평균 수분으로 나눠 상대적인 수분을 나타냄)
- Mulch_moisture_interaction

3. 범주형 처리

- Train/test를 합쳐서 카테고리 전체 목록을 만든 후 양쪽 모두 동일한 categories를 갖게 변환(인코딩 불일치 방지용)

4. 타겟 인코딩

5. 하이퍼파라미터

- Optuna로 미리 튜닝된 값을 디렉터리로 하드코딩함.

6. 5-Fold CV 및 3개 모델 학습

- StratifiedKFold(5)로 폴드를 나누고, 폴드마다 lightgbm, catboost, xgboost 모델을 각각 학습함.

7. 스택킹

- 3개 모델의 OOF 확률을 이어붙여서 메타 피쳐를 구성하고, LogisticRegression을 메타 모델로 학습함.

<주요 코드>

1. train에서 만든 매핑을 데이터 누수 방지용으로 test에 동일하게 사용

```
1 if train_moisture_map is None:
2     train_moisture_map = df.groupby('Crop_Type')['Soil_Moisture'].mean()
3 df['Relative_Crop_Moisture'] = df['Soil_Moisture'] / df['Crop_Type'].map(train_moisture_map)
```

2. out-of-fold 예측을 검증 인덱스 위치에만 채워 넣어 스택킹용 메타 피처를 안전하게 생성함

```
1 m_lgb = lgb.LGBMClassifier(device='gpu', class_weight='balanced', **best_lgb_params, verbose=-1)
2 m_lgb.fit(xt, yt)
3 oof_lgb[v_idx] = m_lgb.predict_proba(xv)
4 test_lgb += m_lgb.predict_proba(X_test) / 5
```

3. 세 모델의 OOF 확률을 이어붙여 메타 피처를 구성하고 로지스틱 회귀로 스택킹함.

```
1 X_meta_train = np.hstack([oof_lgb, oof_cat, oof_xgb])
2 meta_model = LogisticRegression(class_weight='balanced', max_iter=2000)
3 meta_model.fit(X_meta_train, y_train)
```

<배운점>

Test/train을 합쳐서 전체 카테고리 목록을 먼저 만들고, 양쪽에 동일하게 적용하는 방식이 원-핫 인코딩시 흔히 발생하는 train에 없던 카테고리가 test에 나타나서 칼럼이 어긋나는 문제를 트리 기반 모델에서는 이렇게 깔끔하게 처리할 수 있음을 배움

<어려운 점>

optuna과정이 노트북에 없고 결과값만 있어서 실제로 어떤 탐색 범위나 목적함수로 튜닝했는지 알 수 없음.

<새롭게 알게 된 내용>

OOF 기반 스택킹을 실제로 어떻게 구현하는지 알 수 있었음. 결국 학습에 쓰이지 않은 field의 예측(oof)만 모아서 메타모델을 학습해야 데이터 누수가 안생긴다는 것임.